

EPICODE

Report S11/L5

Relazione di Laboratorio — Progetto

Studente: Alessandro Porfirio

Corso: Cisco CyberOps – Epicode

Data: 19 Giugno 2026

Indice

Esercizio 1 — Usare Windows PowerShell

- Parte 1-3: Console PowerShell, comandiecmdlet
- Parte 4: Il comando netstat e i processi
- Parte 5: Svuotare il Cestino con PowerShell

Esercizio 2 — Studio IoC (ANY.RUN)

- Analisi della minaccia e Indicatori di Compromissione

Bonus 1 — Esplorazione di Nmap

- Parte 1: Esplorazione di Nmap (man page)
- Parte 2: Scansione delle porte aperte (localhost, rete, remoto)

Bonus 2 — Attacco a un database MySQL

- Parte 1-6: Analisi dell'attacco SQL Injection con Wireshark
- Domande di riflessione

Esercizio 1: Usare Windows PowerShell

L'obiettivo di questo esercizio era esplorare le funzioni principali di Windows PowerShell: l'accesso alla console, il confronto con il Prompt dei Comandi, l'uso dei cmdlet, l'esplorazione del comando **netstat** e la gestione del Cestino tramite comandi PowerShell.

Parte 1-2: Accedere alla console e confrontare i comandi

Ho aperto sia la console PowerShell sia il Prompt dei Comandi sulla VM Windows (Administrator), ed eseguito lo stesso comando **dir** in entrambe le finestre per confrontarne l'output.

```
PS C:\Users\Administrator> dir

Directory: C:\Users\Administrator

Mode                LastWriteTime         Length Name
----                -
d-r---             6/8/2026  10:21 AM                Contacts
d-r---             6/8/2026  10:21 AM                Desktop
d-r---             6/8/2026  10:21 AM                Documents
d-r---             6/8/2026  10:21 AM                Downloads
d-r---             6/8/2026  10:21 AM                Favorites
d-r---             6/8/2026  10:21 AM                Links
d-r---             6/8/2026  10:21 AM                Music
d-r---             6/8/2026  10:21 AM                Pictures
d-r---             6/8/2026  10:21 AM                Saved Games
d-r---             6/8/2026  10:21 AM                Searches
d-r---             6/8/2026  10:21 AM                Videos
```

Figura 1.1 — Comando dir eseguito in PowerShell: l'output è in formato tabellare con colonne Mode, LastWriteTime, Length, Name.

```
C:\Users\Administrator>dir
Volume in drive C has no label.
Volume Serial Number is 32BD-987A

Directory of C:\Users\Administrator

06/08/2026  10:21 AM    <DIR>          .
06/08/2026  10:21 AM    <DIR>          ..
06/08/2026  10:21 AM    <DIR>          Contacts
06/08/2026  10:21 AM    <DIR>          Desktop
06/08/2026  10:21 AM    <DIR>          Documents
06/08/2026  10:21 AM    <DIR>          Downloads
06/08/2026  10:21 AM    <DIR>          Favorites
06/08/2026  10:21 AM    <DIR>          Links
06/08/2026  10:21 AM    <DIR>          Music
06/08/2026  10:21 AM    <DIR>          Pictures
06/08/2026  10:21 AM    <DIR>          Saved Games
06/08/2026  10:21 AM    <DIR>          Searches
06/08/2026  10:21 AM    <DIR>          Videos
               0 File(s)              0 bytes
               13 Dir(s)  56,916,258,816 bytes free
```

Figura 1.2 — Lo stesso comando dir eseguito nel Prompt dei Comandi: l'output è puramente testuale e include informazioni aggiuntive come il numero di serie del volume e lo spazio libero.

Quali sono gli output del comando dir?

Ho notato una differenza sostanziale tra i due ambienti. In **PowerShell** `dir` è in realtà un alias che richiama il cmdlet `Get-ChildItem`: l'output è strutturato come una vera e propria tabella di oggetti, con colonne `Mode` (attributi tipo `d-r---`), `LastWriteTime`, `Length` e `Name`. Nel **Prompt dei Comandi**, invece, `dir` è un comando nativo che restituisce un output testuale 'piatto', non strutturato come oggetti, ma arricchito con informazioni di sistema come il numero di serie del volume, la dicitura `<DIR>` per le cartelle e il riepilogo finale con il numero di file/cartelle e i byte liberi su disco. Questo conferma la natura object-oriented di PowerShell rispetto all'approccio testuale classico del `cmd`.

Ho poi provato altri comandi già noti dal Prompt dei Comandi, come **ping**, **cd** e **ipconfig**, per verificare se funzionassero anche in PowerShell.

```
C:\Users\Administrator>cd Desktop  
C:\Users\Administrator\Desktop>|
```

Figura 1.3 — Il comando `cd Desktop` funziona identicamente in PowerShell, cambiando la directory corrente in `C:\Users\Administrator\Desktop`.

```
C:\Users\Administrator>ping google.com  
  
Pinging google.com [142.250.181.174] with 32 bytes of data:  
Reply from 142.250.181.174: bytes=32 time=17ms TTL=114  
Reply from 142.250.181.174: bytes=32 time=18ms TTL=114  
Reply from 142.250.181.174: bytes=32 time=21ms TTL=114  
Reply from 142.250.181.174: bytes=32 time=20ms TTL=114  
  
Ping statistics for 142.250.181.174:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
    Approximate round trip times in milli-seconds:  
        Minimum = 17ms, Maximum = 21ms, Average = 19ms
```

Figura 1.4 — Il comando `ping google.com` restituisce 4 risposte da 142.250.181.174, con 0% di pacchetti persi e un RTT medio di 19ms.

```
C:\Users\Administrator>ipconfig  
  
Windows IP Configuration  
  
Ethernet adapter Ethernet:  
  
    Connection-specific DNS Suffix  . :  
    IPv6 Address. . . . . : fd66:d94e:aaea:d021:4b56:4c31:b52d:1b76  
    Link-local IPv6 Address . . . . . : fe80::4b77:79d:c111:d7e4%7  
    IPv4 Address. . . . . : 192.168.60.10  
    Subnet Mask . . . . . : 255.255.255.0  
    Default Gateway . . . . . : 192.168.60.1
```

Figura 1.5 — Il comando `ipconfig` mostra la configurazione di rete: indirizzo IPv4 192.168.60.10, subnet mask 255.255.255.0, gateway 192.168.60.1 e i relativi indirizzi IPv6.

Quali sono i risultati?

Tutti e tre i comandi funzionano in PowerShell esattamente come nel Prompt dei Comandi, perché PowerShell mantiene la compatibilità con i comandi/utility nativi di Windows (eseguibili come ping.exe, ipconfig.exe). Questo conferma che PowerShell non sostituisce completamente cmd, ma lo include e lo estende: posso continuare a usare i comandi tradizionali, e in più ho a disposizione i cmdlet e l'object pipeline tipici di PowerShell.

Parte 3: Esplorare i cmdlet

I cmdlet PowerShell seguono la convenzione Verbo-Nome. Per scoprire qual è il cmdlet 'reale' dietro l'alias dir, ho usato il comando Get-Alias dir.

```
PS C:\Users\Administrator> Get-Alias dir

CommandType      Name                                Version      S
-----
Alias             dir -> Get-ChildItem
```

Figura 1.6 — Il comando Get-Alias dir conferma che dir è un alias del cmdlet Get-ChildItem.

Qual è il comando PowerShell per dir?

Il cmdlet reale è **Get-ChildItem**: dir è semplicemente un alias creato per facilitare la transizione degli utenti abituati al Prompt dei Comandi. Get-ChildItem segue la convenzione verbo-nome (Get = recupera, ChildItem = elementi figli di un contenitore) ed è il cmdlet che effettivamente elenca file e sottocartelle.

Parte 4: Esplorare il comando netstat usando PowerShell

Ho usato netstat -h per vedere l'elenco delle opzioni disponibili. Nel mio ambiente l'output ha mostrato lo 'Socket Handle Count' (numero di handle di socket aperti per ciascun PID), utile per individuare processi che aprono un numero anomalo di connessioni di rete.

```
PS C:\Users\Administrator> netstat -h

Socket Handle Count

   PID      Count  Closing Count
   ----      -
   1576         6           0
   5168         2           0
   3636         4           0
   3652        38           0
   3664         9           0
   1364         1           0
   1876         4           0
   3672       5021           0
   1120        13           0
   1380         4           0
   1636         2           0
   1140         2           0
   3704         1           0
    900         4           0
    908        661           0
   2744         4           0
   3788         4           0
   2260         4           0
    756         4           0
```

Figura 1.7 — netstat -h: conteggio degli handle socket per PID. Si notano valori elevati per il PID 3672 (5021 handle) e per il PID 908 (661 handle), da tenere d'occhio in un'analisi di sicurezza.

Successivamente ho usato netstat -r per visualizzare la tabella di routing con le rotte attive del mio host.

```
PS C:\Users\Administrator> netstat -r
=====
Interface List
7...1a 4f 46 a8 e8 ea .....Red Hat VirtIO Ethernet Adapter
1.....Software Loopback Interface 1
=====

IPv4 Route Table
=====
Active Routes:
Network Destination        Netmask          Gateway          Interface        Metric
0.0.0.0                    0.0.0.0          192.168.60.1     192.168.60.10    271
127.0.0.0                  255.0.0.0        On-link          127.0.0.1        331
127.0.0.1                  255.255.255.255  On-link          127.0.0.1        331
127.255.255.255            255.255.255.255  On-link          127.0.0.1        331
192.168.60.0                255.255.255.0    On-link          192.168.60.10    271
192.168.60.10              255.255.255.255  On-link          192.168.60.10    271
192.168.60.255             255.255.255.255  On-link          192.168.60.10    271
224.0.0.0                  240.0.0.0        On-link          127.0.0.1        331
224.0.0.0                  240.0.0.0        On-link          192.168.60.10    271
255.255.255.255            255.255.255.255  On-link          127.0.0.1        331
255.255.255.255            255.255.255.255  On-link          192.168.60.10    271
=====
Persistent Routes:
Network Address          Netmask    Gateway Address  Metric
0.0.0.0                  0.0.0.0    192.168.60.1     Default
=====

IPv6 Route Table
=====
Active Routes:
If Metric Network Destination      Gateway
1 331 ::1/128                  On-link
7 271 fd66:d94e:aaea:d021::/64 On-link
7 271 fd66:d94e:aaea:d021:4b56:4c31:b52d:1b76/128
7 271 fe80::/64                On-link
7 271 fe80::4b77:79d:c111:d7e4/128
1 331 ff00::/8                 On-link
7 271 ff00::/8                 On-link
=====
Persistent Routes:
None
=====
```

Figura 1.8 — netstat -r: tabella di routing IPv4/IPv6. L'interfaccia attiva è 192.168.60.10 con gateway di default 192.168.60.1.

Qual è il gateway IPv4?

Il gateway IPv4 della mia VM è **192.168.60.1**, coerente con quanto già osservato con ipconfig (rete 192.168.60.0/24, interfaccia 192.168.60.10).

Ho poi aperto una seconda PowerShell con privilegi elevati (Esegui come amministratore) ed eseguito netstat -abno per visualizzare le connessioni TCP attive insieme ai PID dei processi che le hanno create.

```
PS C:\WINDOWS\system32> netstat -abno
Active Connections
Proto Local Address           Foreign Address         State       PID
TCP 0.0.0.0:88              0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:135             0.0.0.0:0               LISTENING   1120
RpcEptMapper
[svchost.exe]
TCP 0.0.0.0:389            0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:445            0.0.0.0:0               LISTENING   4
Can not obtain ownership information
TCP 0.0.0.0:464            0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:593           0.0.0.0:0               LISTENING   1120
RpcEptMapper
[svchost.exe]
TCP 0.0.0.0:636           0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:3268          0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:3269          0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:5985          0.0.0.0:0               LISTENING   4
Can not obtain ownership information
TCP 0.0.0.0:9389          0.0.0.0:0               LISTENING   3652
[Microsoft.ActiveDirectory.WebServices.exe]
TCP 0.0.0.0:47001         0.0.0.0:0               LISTENING   4
Can not obtain ownership information
TCP 0.0.0.0:49664         0.0.0.0:0               LISTENING   908
[lsass.exe]
TCP 0.0.0.0:49665         0.0.0.0:0               LISTENING   756
Can not obtain ownership information
TCP 0.0.0.0:49668         0.0.0.0:0               LISTENING   1876
EventLog
[svchost.exe]
TCP 0.0.0.0:49669        0.0.0.0:0               LISTENING   2260
Schedule
[svchost.exe]
TCP 0.0.0.0:49670        0.0.0.0:0               LISTENING   2744
[spoolsv.exe]
```

Figura 1.9 — netstat -abno: connessioni TCP in ascolto con il relativo PID e nome processo (es. lsass.exe sulla porta 88/389/445/636, svchost.exe sulla porta 135/593, Microsoft.ActiveDirectory.WebServices.exe sulla 9389). Trattandosi di un Domain Controller, è normale vedere lsass.exe in ascolto su molte porte LDAP/Kerberos.

Ho poi aperto Gestione Attività (Task Manager), navigato alla scheda Dettagli e ordinato le colonne per PID, per individuare visivamente i processi rilevati da netstat. Ho anche usato il cmdlet Get-Process come equivalente PowerShell del Task Manager, per ottenere lo stesso tipo di informazioni (PID, nome processo, consumo CPU) direttamente da riga di comando.

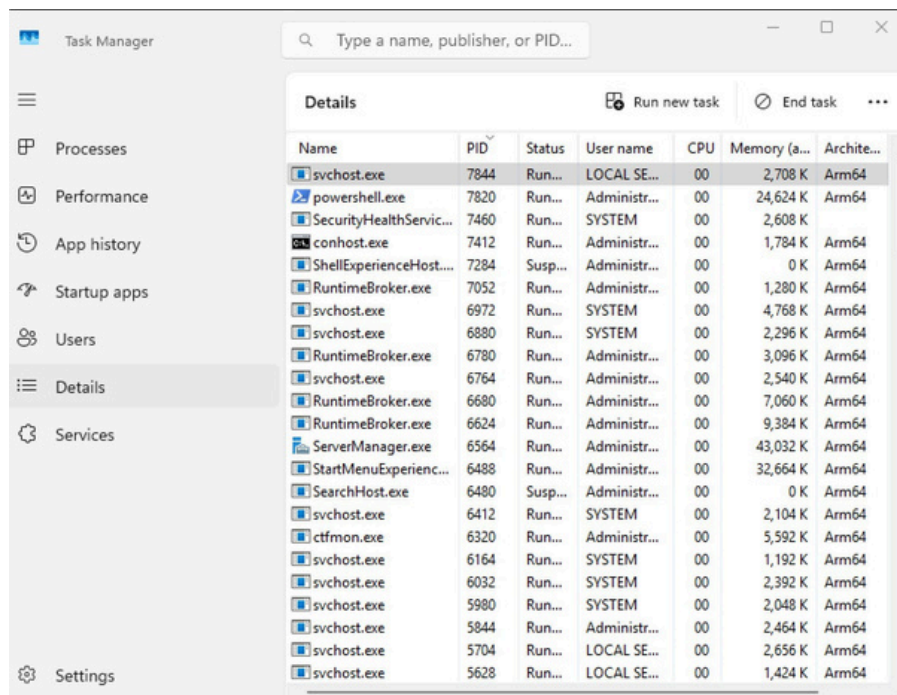


Figura 1.10 — Task Manager, scheda Dettagli, ordinata per PID. La colonna Architettura conferma che si tratta di processi ARM64, coerente con l'ambiente UTM su Apple Silicon usato per il laboratorio.

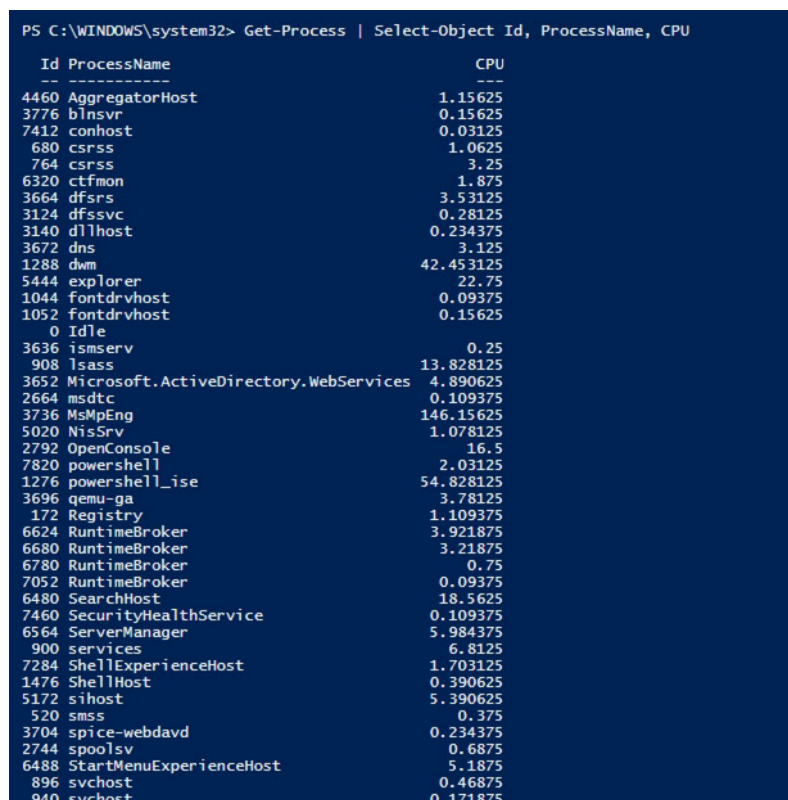


Figura 1.11 — Equivalente PowerShell: Get-Process | Select-Object Id, ProcessName, CPU. Si nota che MsMpEng (Windows Defender) e powershell_ise hanno il maggior consumo di CPU cumulativo tra i processi visibili.

Ho selezionato uno dei PID individuati con netstat (in particolare un processo svchost.exe) e ho aperto la finestra di dialogo Proprietà dal relativo eseguibile per ottenere maggiori dettagli.

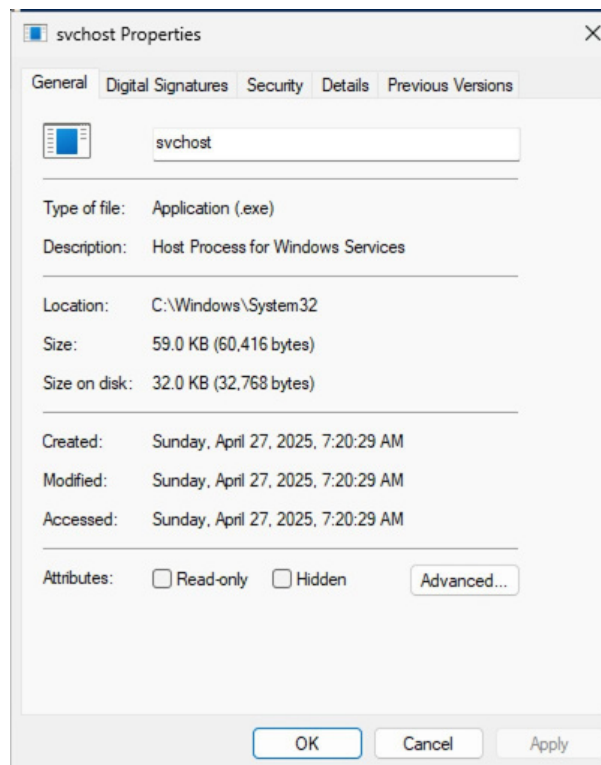


Figura 1.12 — Finestra Proprietà di svchost.exe: tipo file Application (.exe), descrizione 'Host Process for Windows Services', percorso C:\Windows\System32, dimensione 59,0 KB.

Quali informazioni puoi ottenere dalla scheda Dettagli e dalla finestra di dialogo Proprietà per il PID selezionato?

Dalla scheda **Dettagli** di Task Manager ottengo informazioni operative in tempo reale: PID, stato del processo (in esecuzione/sospeso), nome utente associato (SYSTEM, LOCAL SERVICE, Administrator...), percentuale di CPU, memoria utilizzata e architettura (nel mio caso Arm64). Dalla finestra **Proprietà** dell'eseguibile ottengo invece informazioni 'statiche' relative al file stesso: tipo di file, descrizione del produttore, percorso completo sul disco, dimensione su disco e date di creazione/modifica/accesso, oltre alle schede Firme digitali e Sicurezza che permettono di verificare l'autenticità e i permessi del file. Incrociare questi due livelli di informazione è fondamentale in un'analisi di sicurezza: un processo che si chiama svchost.exe ma che NON si trova in C:\Windows\System32, o che non ha una firma digitale Microsoft valida, è un forte indicatore di un eseguibile malevolo che si maschera da processo legittimo (masquerading).

Parte 5: Svuotare il Cestino usando PowerShell

Per dimostrare come PowerShell semplifichi attività che richiederebbero più passaggi tramite l'interfaccia grafica, ho usato il cmdlet Clear-RecycleBin per svuotare il Cestino in un solo comando.

```
PS C:\WINDOWS\system32> Clear-RecycleBin
```

Figura 1.13 — Esecuzione del cmdlet Clear-RecycleBin da una PowerShell con privilegi elevati.

Cosa è successo ai file nel Cestino?

Tutti i file presenti nel Cestino sono stati **eliminati in modo permanente** dal disco, senza possibilità di ripristino tramite l'interfaccia grafica. Il comando `Clear-RecycleBin` richiede normalmente una conferma (Y/N) prima di procedere, esattamente come l'azione 'Svuota cestino' da interfaccia grafica, ma può essere eseguito anche con il flag `-Confirm:$false` per saltare la richiesta di conferma in automazione, ad esempio all'interno di uno script di manutenzione pianificato.

Domanda di Riflessione — Comandi PowerShell per l'automazione delle attività di sicurezza

Ricercando su internet ho individuato diversi comandi/cmdlet particolarmente utili per un analista di sicurezza: **Get-WinEvent** e **Get-EventLog** per interrogare i log di sicurezza di Windows e cercare eventi sospetti (es. logon falliti, EventID 4625); **Get-Service** e **Stop-Service/Start-Service** per verificare e controllare lo stato dei servizi su più macchine contemporaneamente; **Get-NetTCPConnection**, evoluzione object-oriented di netstat, che permette di filtrare le connessioni con `Where-Object`; **Get-FileHash** per calcolare velocemente l'hash (MD5/SHA256) di un file sospetto e confrontarlo con i database di Threat Intelligence; e infine **Invoke-Command**, che insieme al PowerShell Remoting (WinRM) consente di eseguire comandi di verifica o di remediation su decine di host contemporaneamente, fondamentale durante un incident response su larga scala.

Esercizio 2: Studio IoC

Per questo esercizio ho analizzato il task pubblico su ANY.RUN indicato nella traccia, una sandbox online che esegue file/URL sospetti in un ambiente controllato (Win10 64bit) e ne registra il comportamento (processi, traffico di rete, modifiche al filesystem). Di seguito il mio piccolo report sulle minacce osservate.

Contesto dell'analisi

Il task analizzato mostra la navigazione, all'interno della sandbox, verso due repository GitHub del medesimo autore ("MELITERRER"): **MELITERRER/kioluu** (file Muadnrd.exe) e **MELITERRER/frew** (file Jvczfhe.exe). ANY.RUN classifica l'intera sessione come **Malicious activity**, con un punteggio molto alto (94/100) associato al processo firefox.exe che ha effettuato il download.

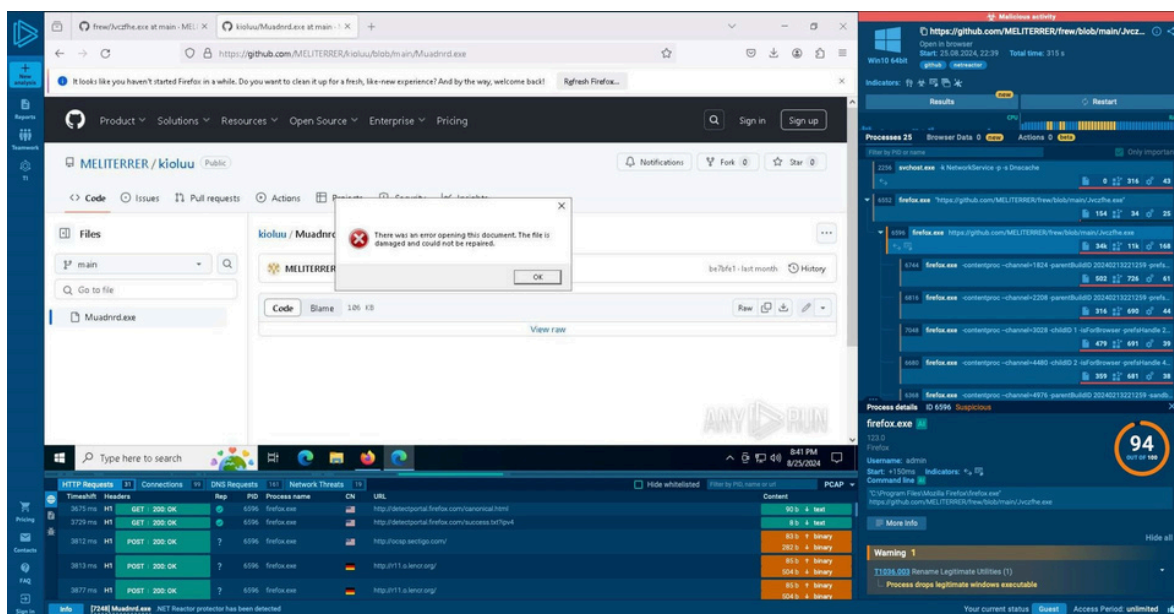


Figura 2.1 — Vista generale del task ANY.RUN: banner rosso 'Malicious activity' in alto, repository GitHub MELITERRER/kioluu aperto nel browser con un finto 'errore di apertura documento' (tecnica usata per indurre l'utente a riprovare il download), albero dei processi a destra con firefox.exe (score 94/100) e numerosi processi figli -contentproc, e tab Network Threats in basso che mostra query DNS verso domini *.duckdns.org.

Indicatore 1 — Distribuzione del malware tramite GitHub

L'attore malevolo ospita gli eseguibili direttamente su GitHub, sfruttando la reputazione del dominio github.com per superare filtri web e proxy aziendali che tipicamente non bloccano questo dominio. Il file viene presentato come un documento/eseguibile da scaricare; al tentativo di apertura nel browser appare un messaggio fittizio di 'documento danneggiato', tecnica comune per spingere l'utente a scaricare ed eseguire manualmente il file invece di limitarsi alla sola visualizzazione.

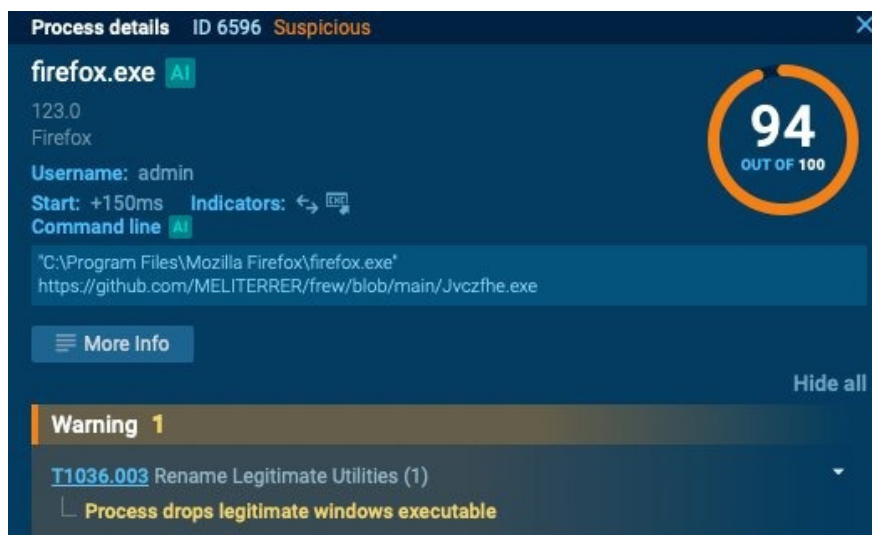


Figura 2.2 — Dettaglio del processo firefox.exe (ID 6596): command line che apre direttamente l'URL github.com/MELITERRER/frew/blob/main/Jvczfhe.exe. Viene segnalata la tecnica MITRE ATT&CK; T1036.003 'Rename Legitimate Utilities': il processo lascia cadere (drops) un eseguibile Windows legittimo rinominato, tipica tecnica di evasione/persistenza.

Indicatore 2 — Processo Muadnrd.exe e tecniche di evasione

Il file scaricato dal primo repository, Muadnrd.exe, viene eseguito dalla cartella Downloads dell'utente (C:\Users\admin\Downloads\Muadnrd.exe), posizione tipica per i payload di prima esecuzione. ANY.RUN gli assegna un punteggio di 62/100 (Suspicious) e segnala due comportamenti di warning: il processo si auto-rilancia ('Application launched itself', spesso usato per staccarsi dal processo padre o rilanciarsi con argomenti diversi) e include un'applicazione che va in crash ('Executes application which crashes'), comportamento compatibile con un payload che tenta più tecniche di esecuzione/iniezione fino a trovarne una funzionante sull'host.

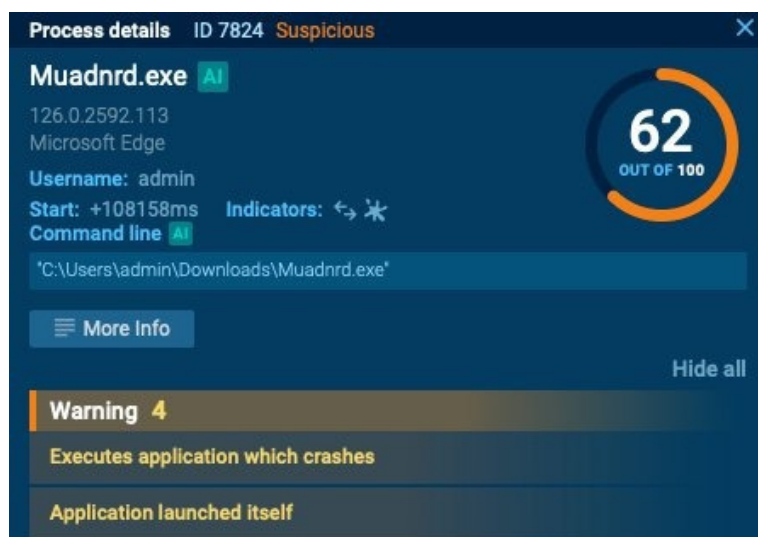
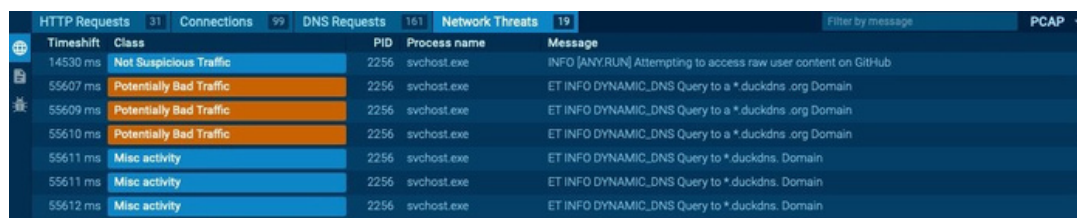


Figura 2.3 — Dettaglio del processo Muadnrd.exe (ID 7824, score 62/100 Suspicious): eseguito dalla cartella Downloads, con warning 'Application launched itself' ed 'Executes application which crashes'.

Nella scheda HTTP Requests della sandbox è inoltre visibile l'etichetta '**.NET Reactor protector has been detected**' riferita al file Muadnrd.exe: .NET Reactor è un packer/obfuscator legittimo usato però molto spesso dal malware per ostacolare l'analisi statica e l'estrazione delle stringhe, rendendo difficile la scansione da parte degli antivirus tradizionali basati su firme.

Indicatore 3 — Traffico di rete e C2 su Dynamic DNS

La parte più significativa dal punto di vista di Threat Intelligence è il traffico di rete generato durante l'esecuzione: la sandbox registra ripetute query DNS verso sottodomini di ***.duckdns.org**, classificate da ANY.RUN come 'Potentially Bad Traffic'.



Timeshift	Class	PID	Process name	Message
14530 ms	Not Suspicious Traffic	2256	svchost.exe	INFO [ANY.RUN] Attempting to access raw user content on GitHub
55607 ms	Potentially Bad Traffic	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to a *.duckdns.org Domain
55609 ms	Potentially Bad Traffic	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to a *.duckdns.org Domain
55610 ms	Potentially Bad Traffic	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to a *.duckdns.org Domain
55611 ms	Misc activity	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to *.duckdns. Domain
55611 ms	Misc activity	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to *.duckdns. Domain
55612 ms	Misc activity	2256	svchost.exe	ET INFO DYNAMIC_DNS Query to *.duckdns. Domain

Figura 2.4 — Tab Network Threats: query DNS ripetute verso domini dinamici *.duckdns.org, etichettate 'ET INFO DYNAMIC_DNS Query' e classificate come Potentially Bad Traffic.

DuckDNS è un servizio gratuito di Dynamic DNS, legittimo se usato per scopi personali (es. accesso remoto a un homelab), ma è anche uno dei servizi più sfruttati dai gruppi malware per ospitare l'infrastruttura di Command & Control (C2), proprio perché permette di cambiare rapidamente l'IP a cui punta il dominio senza dover registrare un nuovo dominio ogni volta, rendendo più difficile il blacklisting permanente da parte dei difensori. Una query DNS dinamica e ripetuta verso *.duckdns.org generata da un processo appena scaricato da GitHub è un IoC molto forte di comunicazione C2 attiva.

Conclusioni dell'analisi

Il verdetto assegnato da ANY.RUN al task è **Malicious activity** (tag: github, netreactor), ma è importante essere precisi: nella sezione 'Behavior activities' del report ufficiale, la categoria a gravità massima (MALICIOUS) riporta esplicitamente '**No malicious indicators**' — su 155 processi totali monitorati, 0 sono classificati come malicious e solo 3 come suspicious. Il verdetto complessivo deriva quindi dalla combinazione di più segnali di livello SUSPICIOUS/INFO piuttosto che da un singolo comportamento dannoso conclamato. Gli indicatori concreti raccolti sono comunque coerenti con un **dropper/loader in fase di staging**: scaricato via repository GitHub (MELITERRER/kioluu e MELITERRER/frew), aspettando del tempo per poi procedere con l'offuscamento con .NET Reactor, abuso del LOLBin InstallUtil.exe per l'esecuzione di codice .NET, tecniche di evasione (disattivazione log ETW, ritardo con timeout.exe, controllo delle impostazioni di trust di Windows) e traffico DNS sospetto verso Dynamic DNS (duckdns.org, attribuzione indiretta tramite il servizio di sistema Dnscache). Gli IoC da monitorare/bloccare restano:

hash MD5 00B5E91B42712471CDFBDB37B715670C (Jvczfhe.exe, SHA256 0307EE805DF8B94733598D5C3D62B28678EAEADBF1CA3689FA678A3780DD3DF0), i repository github.com/MELITERRER/kioluu e github.com/MELITERRER/frew, ed eventuale traffico DNS anomalo verso *.duckdns.org.

Bonus 1: Esplorazione di Nmap

In questo laboratorio bonus ho esplorato Nmap, una delle utility più importanti per la scoperta della rete e l'audit di sicurezza, partendo dalla consultazione della sua pagina man fino alla scansione di host locali, della rete locale e di un server remoto autorizzato.

Parte 1: Esplorazione di Nmap

Ho avviato la VM CyberOps Workstation e, da terminale, ho consultato la pagina manuale con `man nmap`.

```
NMAP(1)                                Nmap Reference Guide                                NMAP(1)

NAME
  nmap - Network exploration tool and security / port scanner

SYNOPSIS
  nmap [Scan Type...] [Options] {target specification}

DESCRIPTION
  Nmap ("Network Mapper") is an open source tool for network exploration
  and security auditing. It was designed to rapidly scan large networks,
  although it works fine against single hosts. Nmap uses raw IP packets in
  novel ways to determine what hosts are available on the network, what
  services (application name and version) those hosts are offering, what
  operating systems (and OS versions) they are running, what type of
  packet filters/firewalls are in use, and dozens of other
  characteristics. While Nmap is commonly used for security audits, many
  systems and network administrators find it useful for routine tasks such
  as network inventory, managing service upgrade schedules, and monitoring
  host or service uptime.

  The output from Nmap is a list of scanned targets, with supplemental
  information on each depending on the options used. Key among that
  information is the "interesting ports table". That table lists the port
  number and protocol, service name, and state. The state is either open,
  filtered, closed, or unfiltered. Open means that an application on the
  target machine is listening for connections/packets on that port.
  Filtered means that a firewall, filter, or other network obstacle is
  blocking the port so that Nmap cannot tell whether it is open or closed.
  Closed ports have no application listening on them, though they could
  open up at any time. Ports are classified as unfiltered when they are
  responsive to Nmap's probes, but Nmap cannot determine whether they are
  open or closed. Nmap reports the state combinations open|filtered and
  closed|filtered when it cannot determine which of the two states
  describe a port. The port table may also include software version
  details when version detection has been requested. When an IP protocol
  scan is requested (-s0), Nmap provides information on supported IP
  protocols rather than listening ports.
```

Figura 3.1 — Sezioni NAME, SYNOPSIS e DESCRIPTION della pagina man di Nmap: viene descritto come 'Network exploration tool and security / port scanner', e si spiegano gli stati di una porta (open, filtered, closed, unfiltered).

Cos'è Nmap?

Nmap (Network Mapper) è un tool open source per l'esplorazione di rete e l'audit di sicurezza. Usa pacchetti IP grezzi per determinare quali host sono attivi sulla rete, quali servizi (nome e versione dell'applicazione) offrono, quale sistema operativo (e relativa versione) stanno eseguendo, che tipo di filtri/firewall sono in uso e numerose altre caratteristiche della rete target.

Per cosa viene usato nmap?

Sebbene sia stato progettato per scansionare rapidamente grandi reti, funziona bene anche contro singoli host. Viene usato principalmente per audit di sicurezza, ma anche per attività di amministrazione di sistema ordinarie come l'inventario di rete, la gestione dei piani di aggiornamento dei servizi e il monitoraggio dell'uptime di host e servizi.

Ho poi usato la funzione di ricerca della pagina man (/example) per individuare la sezione 'Example 1' che mostra un comando rappresentativo di Nmap.

```
NMAP(1)                                Nmap Reference Guide                                NMAP(1)

NAME
    nmap - Network exploration tool and security / port scanner

SYNOPSIS
    nmap [Scan Type...] [Options] {target specification}

DESCRIPTION
    Nmap ("Network Mapper") is an open source tool for network exploration and security auditing. It was designed to rapidly scan large networks, although it works fine against single hosts. Nmap uses raw IP packets in novel ways to determine what hosts are available on the network, what services (application name and version) those hosts are offering, what operating systems (and OS versions) they are running, what type of packet filters/firewalls are in use, and dozens of other characteristics. While Nmap is commonly used for security audits, many systems and network administrators find it useful for routine tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime.

    The output from Nmap is a list of scanned targets, with supplemental information on each depending on the options used. Key among that information is the "interesting ports table". That table lists the port number and protocol, service name, and state. The state is either open, filtered, closed, or unfiltered. Open means that an application on the target machine is listening for connections/packets on that port. Filtered means that a firewall, filter, or other network obstacle is blocking the port so that Nmap cannot tell whether it is open or closed. Closed ports have no application listening on them, though they could open up at any time. Ports are classified as unfiltered when they are responsive to Nmap's probes, but Nmap cannot determine whether they are open or closed. Nmap reports the state combinations open|filtered and closed|filtered when it cannot determine which of the two states describe a port. The port table may also include software version details when version detection has been requested. When an IP protocol scan is requested (-s0), Nmap provides information on supported IP protocols rather than listening ports.

    In addition to the interesting ports table, Nmap can provide further information on targets, including reverse DNS names, operating system guesses, device types, and MAC addresses.

    A typical Nmap scan is shown in Example 1. The only Nmap arguments used in this example are -A, to enable OS and version detection, script scanning, and traceroute; -T4 for faster execution; and then the hostname.

    Example 1. A representative Nmap scan

    # nmap -A -T4 scanme.nmap.org

    Nmap scan report for scanme.nmap.org (74.207.244.221)
    Host is up (0.029s latency).
    rDNS record for 74.207.244.221: li86-221.members.linode.com
    Not shown: 995 closed ports
    PORT      STATE      SERVICE      VERSION
    Manual page nmap(1) line 1 (press h for help or q to quit)
```

Figura 3.2 — Ricerca del termine 'example' nella pagina man: viene evidenziata la frase che introduce l'Esempio 1, con gli argomenti -A (rilevamento OS/versione, script scanning, traceroute) e -T4 (esecuzione più rapida).

```
Example 1. A representative Nmap scan

# nmap -A -T4 scanme.nmap.org

Nmap scan report for scanme.nmap.org (74.207.244.221)
Host is up (0.029s latency).
rDNS record for 74.207.244.221: li86-221.members.linode.com
Not shown: 995 closed ports
PORT      STATE      SERVICE      VERSION
22/tcp    open      ssh          OpenSSH 5.3p1 Debian 3ubuntu7 (protocol 2.0)
| ssh-hostkey: 1024 8d:60:f1:7c:ca:b7:3d:0a:d6:67:54:9d:69:d9:b9:dd (DSA)
|_2048 79:f8:09:ac:d4:e2:32:42:10:49:d3:bd:20:82:85:ec (RSA)
80/tcp    open      http         Apache httpd 2.2.14 ((Ubuntu))
|_http-title: Go ahead and ScanMe!
646/tcp   filtered  ldp
1720/tcp  filtered  H.323/Q.931
9929/tcp  open      nping-echo   Nping echo
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.39
OS details: Linux 2.6.39
Network Distance: 11 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:kernel
```

Figura 3.3 — Output completo dell'Esempio 1 (nmap -A -T4 scanme.nmap.org): porte aperte 22/tcp (ssh), 80/tcp (http), 9929/tcp (nping-echo), porte filtrate 646/tcp e 1720/tcp, e rilevamento del sistema operativo Linux 2.6.x.

Qual è il comando nmap usato?

Il comando mostrato nell'Esempio 1 è **nmap -A -T4 scanme.nmap.org**.

Cosa fa l'opzione -A?

L'opzione -A abilita il rilevamento del sistema operativo e della versione dei servizi, lo script scanning (NSE) e il traceroute, tutto in un'unica opzione 'aggressiva' che restituisce molte più informazioni rispetto a una scansione standard.

Cosa fa l'opzione -T4?

L'opzione -T4 imposta il template di timing su 'Aggressive', velocizzando l'esecuzione della scansione (a scapito di un minimo di accuratezza/discrezione rispetto a timing più lenti come -T2), utile su reti affidabili e veloci dove non serve essere troppo cauti per evitare di sovraccaricare l'host target.

Parte 2: Scansione delle Porte Aperte

Passo 1 — Scansione del localhost

Ho eseguito nmap -A -T4 localhost sulla mia VM CyberOps Workstation.

```
[analyst@secOps ~]$ nmap -A -T4 localhost
Starting Nmap 7.97 ( https://nmap.org ) at 2026-06-19 10:28 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000023s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0 (protocol 2.0)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.28 seconds
```

Figura 3.4 — Scansione di localhost (127.0.0.1): un'unica porta aperta, la 22/tcp con servizio ssh OpenSSH 10.0 (protocol 2.0).

Quali porte e servizi sono aperti? Per ognuna delle porte aperte, registra il software che fornisce i servizi.

Nel mio caso risulta aperta una sola porta: **22/tcp**, servizio **ssh**, fornito dal software **OpenSSH versione 10.0** (protocollo SSH 2.0). Le restanti 999 porte scansionate risultano chiuse (conn-refused), quindi la mia VM espone esclusivamente il servizio SSH, una configurazione minimale e sicura per un host di laboratorio.

Passo 2 — Scansione della rete locale

Prima di scansionare la rete ho ottenuto il mio indirizzo IP e la subnet mask con il comando ip address.

```
[analyst@secOps ~]$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 16:7c:df:f1:fc:de brd ff:ff:ff:ff:ff:ff
   altname enx167cdff1fcde
   inet 192.168.60.2/24 metric 1024 brd 192.168.60.255 scope global dynamic enp0s1
       valid_lft 2920sec preferred_lft 2920sec
   inet6 fe80::147c:dfff:fef1:fcde/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: ovs-system: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether 66:ae:a1:97:c1:95 brd ff:ff:ff:ff:ff:ff
4: s1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether fe:f9:5b:04:bf:4b brd ff:ff:ff:ff:ff:ff
```

Figura 3.5 — Output di ip address: l'interfaccia enp0s1 ha indirizzo 192.168.60.2/24 (subnet mask 255.255.255.0).

Registra l'indirizzo IP e la subnet mask per la tua VM. A quale rete appartiene la tua VM?

Indirizzo IP: **192.168.60.2**, subnet mask: **255.255.255.0** (prefisso /24). La mia VM appartiene quindi alla rete **192.168.60.0/24**.

Ho quindi lanciato `nmap -A -T4 192.168.60.0/24` per individuare gli altri host presenti sulla mia LAN.

```
Nmap scan report for 192.168.60.1 (192.168.60.1)
Host is up (0.00035s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
53/tcp    open  domain (generic dns response: NOTIMP)
5000/tcp  open  rtsp
|_rtsp-methods: ERROR: Script execution failed (use -d to debug)
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/950.7.1
|     X-Apple-ProcessingTime: 7
|     X-Apple-RequestReceivedTimestamp: 232433302
|   GetRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/950.7.1
|     X-Apple-ProcessingTime: 8
|     X-Apple-RequestReceivedTimestamp: 232428264
|   HTTPOptions:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/950.7.1
|     X-Apple-ProcessingTime: 6
|     X-Apple-RequestReceivedTimestamp: 232433288
|   RTSPRequest:
|     RTSP/1.0 403 Forbidden
```

Figura 3.6 — Risultato per l'host 192.168.60.1 (il gateway): porte 53/tcp (domain) e 5000/tcp (rtsp) aperte. Il banner del servizio RTSP (header 'Server: AirTunes/950.7.1') indica che si tratta del gateway/router di casa con AirPlay/Bonjour attivi.

```
7000/tcp  open  rtsp
|_rtsp-methods: ERROR: Script execution failed (use -d to debug)
|_irc-info: Unable to open connection
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/950.7.1
|     X-Apple-ProcessingTime: 6
|     X-Apple-RequestReceivedTimestamp: 232433295
|   GetRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/950.7.1
|     X-Apple-ProcessingTime: 8
|     X-Apple-RequestReceivedTimestamp: 232433266
```

Figura 3.7 — Continuazione della scansione del gateway: anche la porta 7000/tcp (rtsp) risulta aperta, sempre relativa al servizio AirTunes.

```
Nmap scan report for 192.168.60.2 (192.168.60.2)
Host is up (0.00014s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0 (protocol 2.0)
```

Figura 3.8 — Risultato per l'host 192.168.60.2 (la mia stessa VM): porta 22/tcp aperta con OpenSSH 10.0, coerente con la scansione precedente del localhost.

Quanti host sono attivi?

Nella porzione di rete da me scansionata risultano attivi almeno **2 host**: il gateway 192.168.60.1 e la mia VM 192.168.60.2. (Nmap segnala anche eventuali altri host attivi sulla subnet /24, ma questi sono i due principali da me documentati con screenshot).

Dai risultati di Nmap, elenca gli indirizzi IP degli host che si trovano sulla stessa LAN della tua VM. Elenca alcuni dei servizi disponibili sugli host rilevati.

192.168.60.1 (gateway): servizio domain (DNS, porta 53/tcp) e rtsp/AirTunes (porte 5000/tcp e 7000/tcp, tipiche di un router con funzionalità AirPlay).

192.168.60.2 (la mia VM): servizio ssh (porta 22/tcp, OpenSSH 10.0).

Passo 3 — Scansione di un server remoto

Ho aperto il browser e navigato su scanme.nmap.org, il sito ufficiale messo a disposizione dal progetto Nmap per fare pratica di scansione in modo autorizzato.

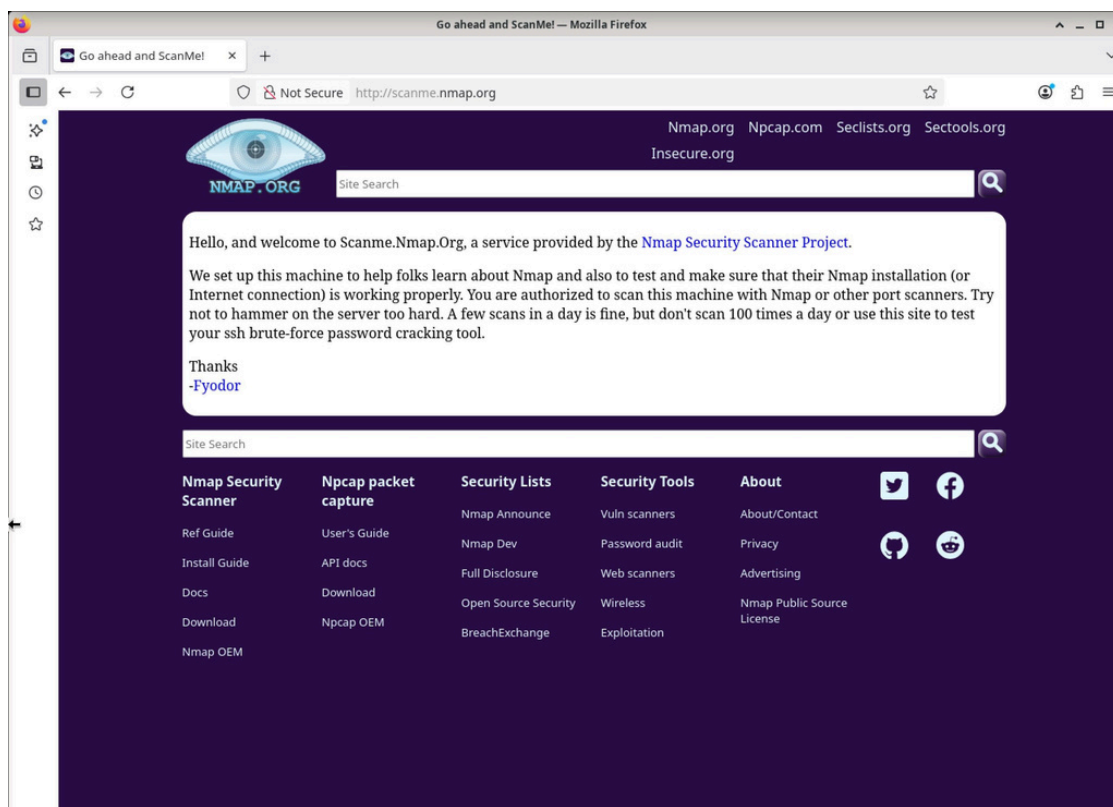


Figura 3.9 — Pagina di benvenuto di scanme.nmap.org, gestita dal progetto Nmap Security Scanner, che autorizza esplicitamente l'uso di Nmap (o altri port scanner) verso questa macchina, a patto di non eccedere con il numero di scansioni giornaliere.

Qual è lo scopo di questo sito?

Scanme.nmap.org è un host messo a disposizione gratuitamente dal progetto Nmap per permettere agli utenti di esercitarsi con le scansioni di rete in modo completamente legale e autorizzato, e per verificare che la propria installazione di Nmap (o la propria connessione a Internet) funzioni correttamente. Il messaggio raccomanda di non abusarne (poche scansioni al giorno, niente brute-force SSH).

Ho quindi eseguito nmap -A -T4 scanme.nmap.org.

```
[analyst@secOps ~]$ nmap -A -T4 scanme.nmap.org
Starting Nmap 7.97 ( https://nmap.org ) at 2026-06-19 10:39 +0000
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.18s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   1024 ac:00:a0:1a:82:ff:cc:55:99:dc:67:2b:34:97:6b:75 (DSA)
|   2048 20:3d:2d:44:62:2a:b0:5a:9d:b5:b3:05:14:c2:a6:b2 (RSA)
|   256 96:02:bb:5e:57:54:1c:4e:45:2f:56:4c:4a:24:b2:57 (ECDSA)
|_  256 33:fa:91:0f:e0:e1:7b:1f:6d:05:a2:b0:f1:54:41:56 (ED25519)
80/tcp    open  http           Apache httpd 2.4.7 ((Ubuntu))
|_ http-favicon: Nmap Project
|_ http-title: Go ahead and ScanMe!
|_ http-server-header: Apache/2.4.7 (Ubuntu)
9929/tcp  open  nping-echo     Nping echo
31337/tcp open  Elite?
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.10 seconds
```

Figura 3.10 — Scansione di scanme.nmap.org (45.33.32.156): porte aperte 22/tcp (ssh, OpenSSH 6.6.1p1 Ubuntu), 80/tcp (http, Apache 2.4.7), 9929/tcp (nping-echo) e 31337/tcp (Elite?); porta 25/tcp filtrata (smtp); sistema operativo Linux.

Quali porte e servizi sono aperti?

22/tcp ssh (OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13), **80/tcp** http (Apache httpd 2.4.7 su Ubuntu, con titolo pagina 'Go ahead and ScanMe!'), **9929/tcp** nping-echo (Nping echo) e **31337/tcp** (servizio 'Elite?', non identificato con certezza da Nmap — 31337 è la classica porta 'eleet' usata storicamente da diversi tool di backdoor/testing).

Quali porte e servizi sono filtrati?

Risulta filtrata la porta **25/tcp (smtp)**: un firewall blocca le richieste su questa porta, e Nmap non riesce a determinare se sia aperta o chiusa.

Qual è l'indirizzo IP del server?

L'indirizzo IP di scanme.nmap.org è **45.33.32.156** (è disponibile anche un indirizzo IPv6: 2600:3c01::f03c:91ff:fe18:bb2f, non scansionato in questo test).

Qual è il sistema operativo?

Il Service Info riportato da Nmap indica **Linux** (cpe:/o:linux:linux_kernel), coerente con le versioni Ubuntu di OpenSSH e Apache rilevate sulle porte aperte.

Domanda di Riflessione — Nmap per la sicurezza della rete

Nmap è uno strumento a doppio taglio. Dal lato **difensivo**, un amministratore o un analista può usarlo per mappare periodicamente la propria rete, individuare porte/servizi esposti non previsti, verificare che le regole firewall funzionino come atteso e identificare versioni software obsolete o vulnerabili prima che lo faccia un attaccante (vulnerability assessment). Dal lato **offensivo**, lo stesso strumento è la base della fase di ricognizione (reconnaissance) di un attacco: un attore malevolo può usare Nmap per mappare silenziosamente l'infrastruttura di un bersaglio, scoprire host raggiungibili da Internet, identificare versioni di servizi con CVE note e pianificare quindi l'exploit più adatto. È per questo che il port scanning non autorizzato è considerato un'attività potenzialmente illecita, ed è fondamentale ottenere sempre il permesso esplicito del proprietario della rete prima di eseguire qualunque scansione, come correttamente indicato nelle istruzioni del laboratorio.

Bonus 2: Attacco a un database MySQL

In quest'ultimo laboratorio ho usato Wireshark per analizzare un file PCAP (SQL_Lab.pcap) contenente la cattura di un attacco reale di SQL Injection contro l'applicazione vulnerabile DVWA (Damn Vulnerable Web Application), seguendo passo passo il flusso TCP/HTTP di ogni richiesta malevola.

Parte 1: Aprire Wireshark e caricare il file PCAP

Ho aperto Wireshark sulla VM CyberOps Workstation e caricato il file SQL_Lab.pcap dalla directory lab.support.files.

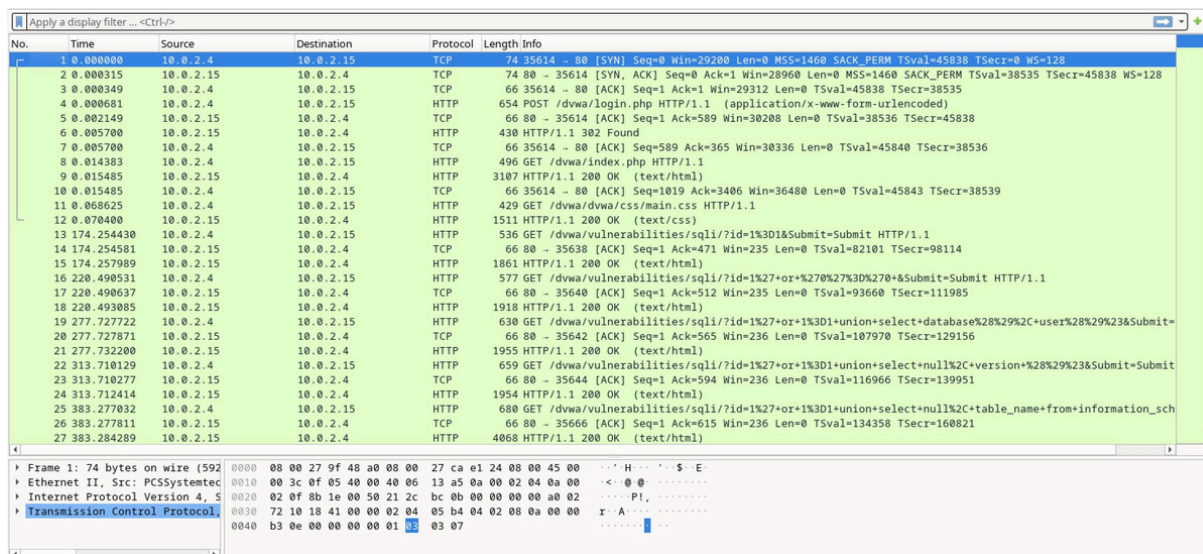


Figura 4.1 — Vista generale della cattura in Wireshark: si nota il three-way handshake TCP iniziale (SYN/SYN-ACK/ACK), una richiesta POST di login a /dvwa/login.php, e una serie di richieste GET verso /dvwa/vulnerabilities/sqli/ con payload SQL via parametro id sempre più articolati.

Quali sono i due indirizzi IP coinvolti in questo attacco di SQL injection?

I due indirizzi IP coinvolti sono **10.0.2.4** (il client/attaccante, che invia le richieste GET/POST malevole) e **10.0.2.15** (il server vittima che ospita l'applicazione DVWA vulnerabile e risponde alle query SQL iniettate).

Parte 2: Visualizzare l'attacco di SQL Injection

Ho selezionato il pacchetto con la prima richiesta GET verso /dvwa/vulnerabilities/sqli/ e usato Segui > Flusso HTTP per analizzare il primo tentativo dell'attaccante, che inserisce 1' OR 1=1 nel campo UserID.

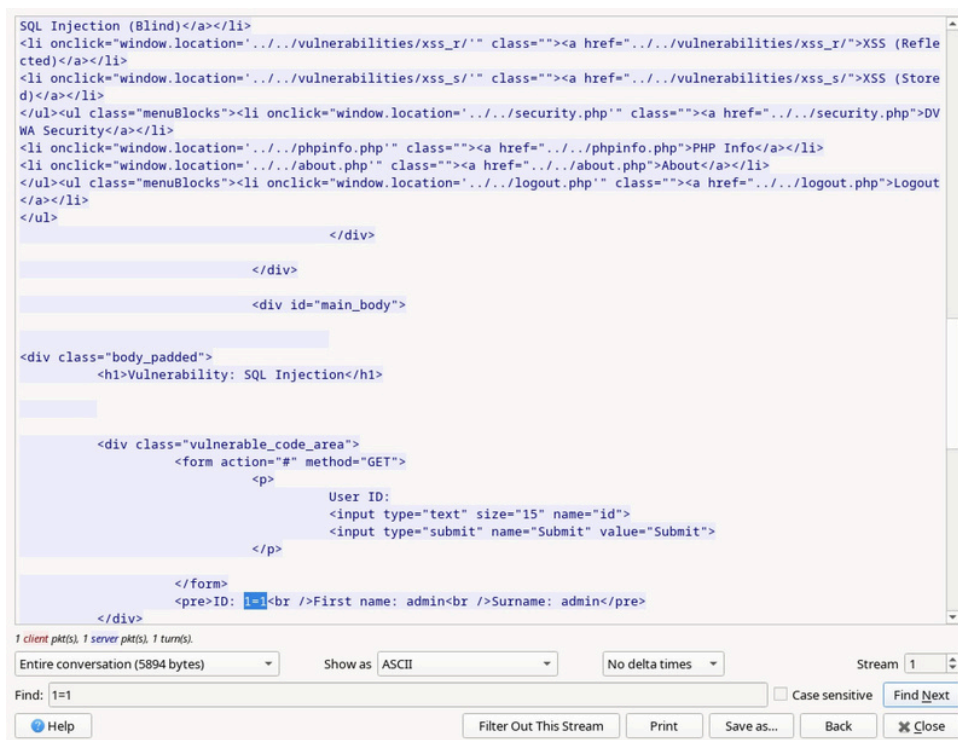


Figura 4.2 — Primo test di SQL Injection (1' or 1=1): l'applicazione risponde con un record valido (ID: 1=1, First name: admin, Surname: admin) invece di un messaggio di errore, confermando che il parametro id è vulnerabile.

La query 1' or 1=1 trasforma la condizione WHERE della query SQL sottostante in una condizione sempre vera, facendo restituire al database un record valido indipendentemente da cosa venga effettivamente cercato: questo è il classico test di conferma di una vulnerabilità SQL Injection.

Parte 3: L'attacco di SQL Injection continua...

Una volta confermata la vulnerabilità, l'attaccante affina la query usando 1' or 1=1 union select database(), user()# per estrarre informazioni sul database stesso tramite una UNION-based SQL Injection.

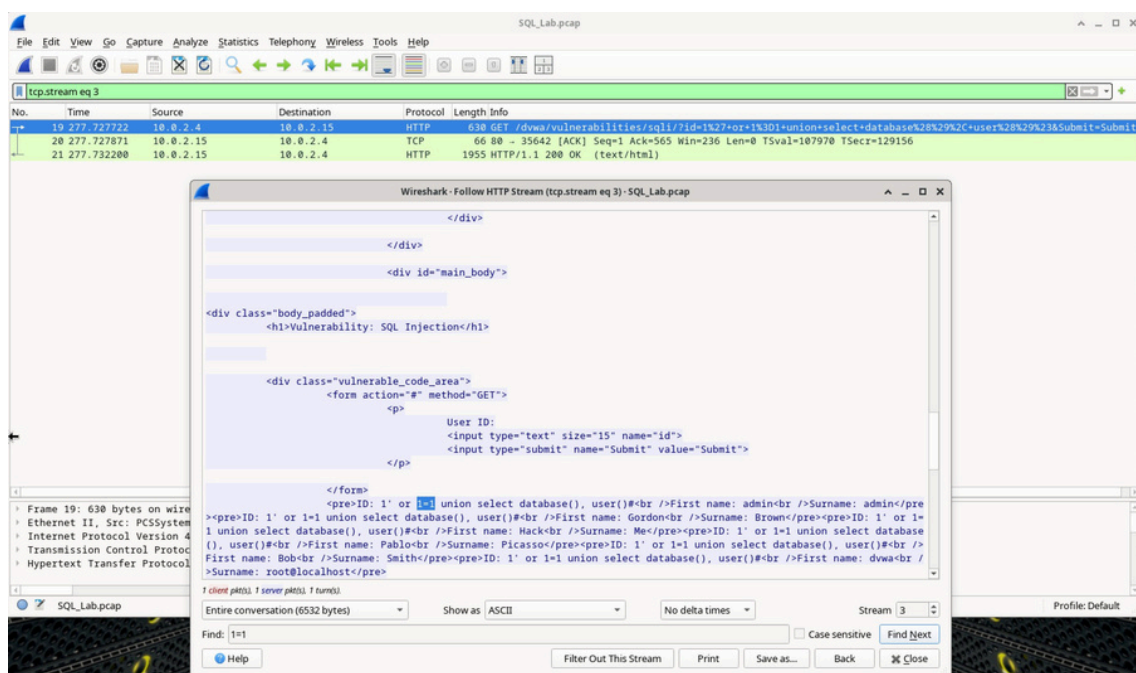


Figura 4.3 — Flusso HTTP (tcp.stream eq 3): la query UNION SELECT database(), user() restituisce per ogni riga il nome del database 'dwva' e l'utente 'root@localhost', oltre ai record legittimi dell'applicazione (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith).

Il database si chiama **dvwa** e l'utente di connessione al database è **root@localhost** — un dettaglio gravissimo dal punto di vista della sicurezza, perché significa che l'applicazione web si connette al database con l'utente amministrativo root invece di un utente con privilegi minimi, amplificando enormemente l'impatto di qualunque SQL Injection riuscita.

Parte 4: L'attacco di SQL Injection fornisce informazioni di sistema

L'attaccante prosegue con `1' or 1=1 union select null, version()#` per individuare la versione esatta del DBMS in uso.

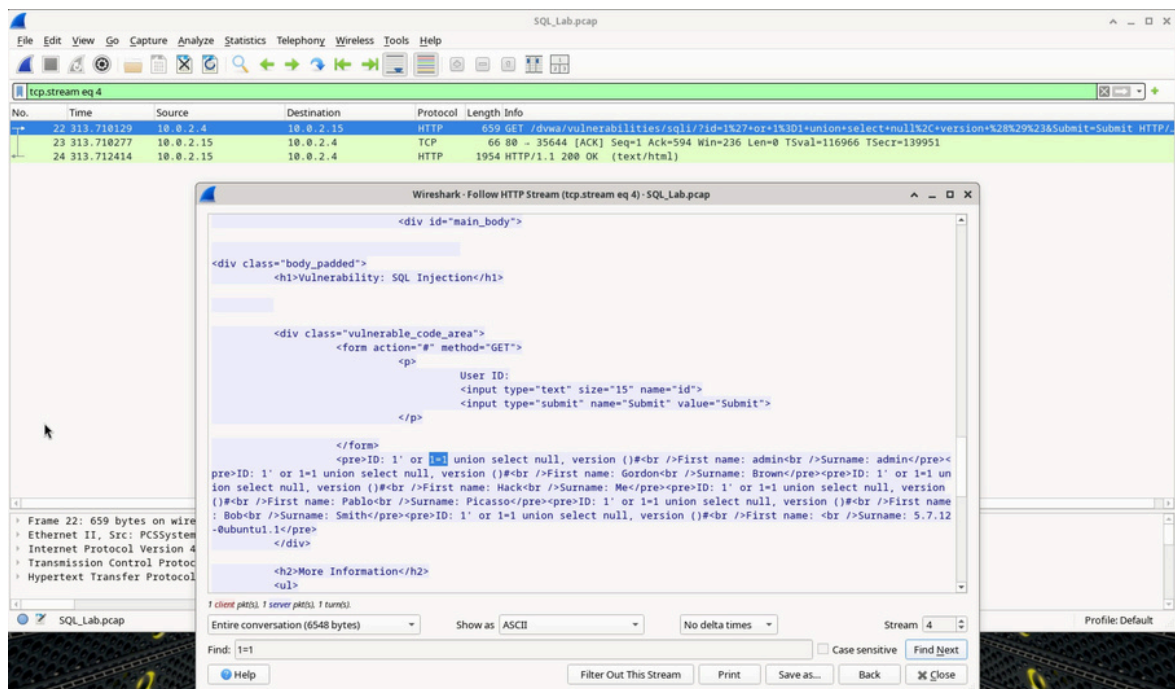


Figura 4.4 — Flusso HTTP (tcp.stream eq 4): la query UNION SELECT null, version() restituisce la stringa di versione subito prima della chiusura del tag `</pre>`.

Qual è la versione?

La versione del DBMS, visibile in fondo all'output prima della chiusura `</pre></div>`, è **5.7.12-0ubuntu1.1** (MySQL 5.7.12 su Ubuntu). Conoscere la versione esatta permette a un attaccante di cercare CVE pubbliche specifiche per quella build, ad esempio vulnerabilità note di privilege escalation o di corruzione di memoria nel motore MySQL stesso.

Parte 5: L'attacco di SQL Injection e le informazioni sulle tabelle

L'attaccante interroga `information_schema.tables` per enumerare tutte le tabelle presenti nel database, con la query `1' or 1=1 union select null, table_name from information_schema.tables#`.

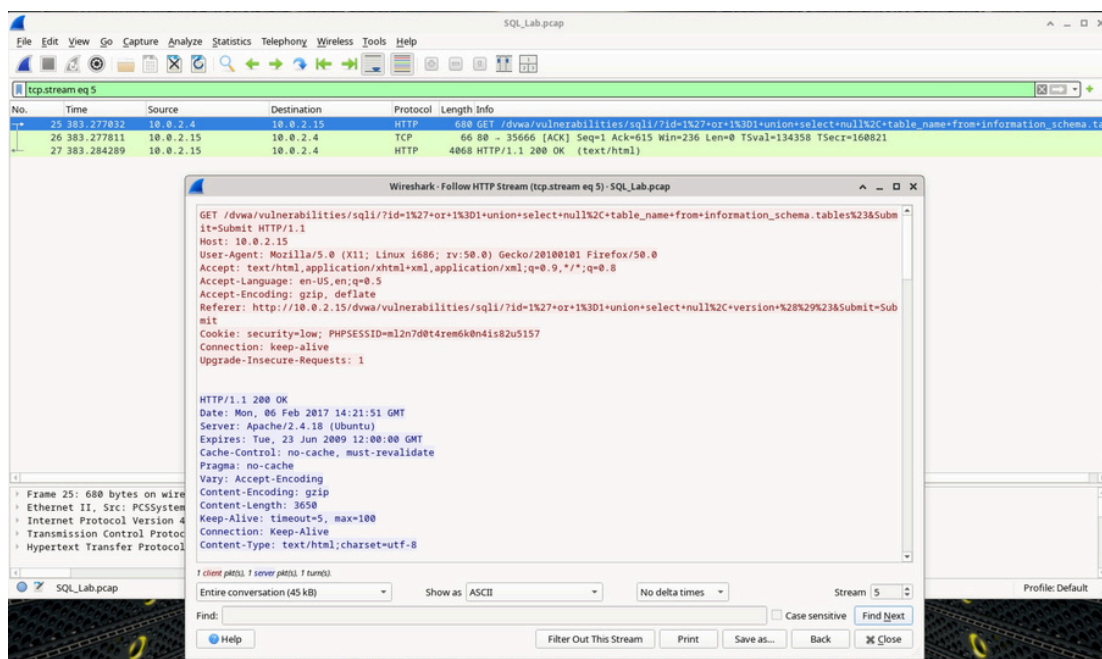


Figura 4.5 — Intestazioni della richiesta (tcp.stream eq 5): GET con la query UNION SELECT su information_schema.tables, header HTTP completi (User-Agent Firefox/50.0, Cookie di sessione PHPSESSID).

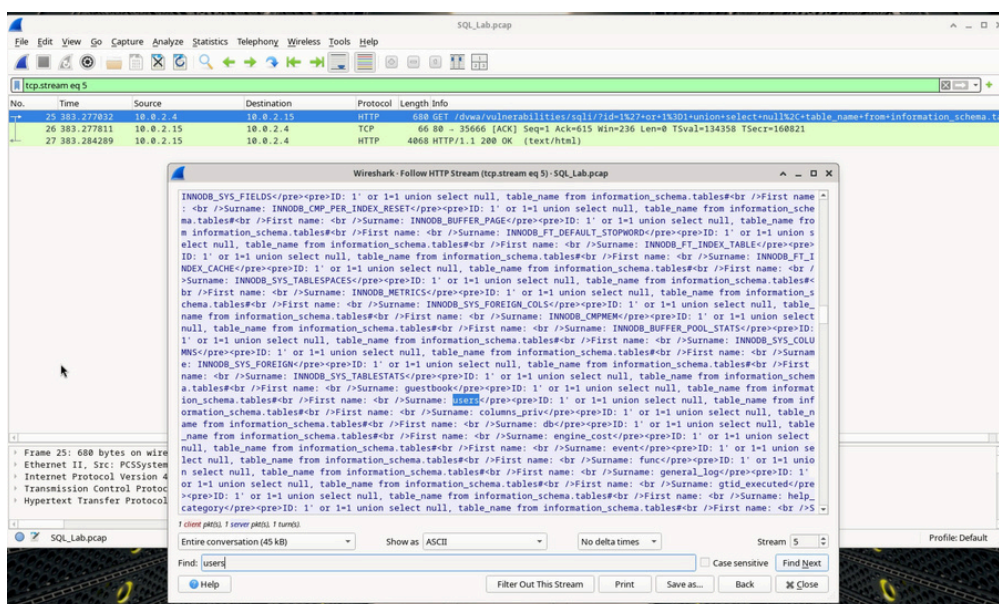


Figura 4.6 — Scorrendo la risposta si osservano decine di tabelle di sistema InnoDB (INNODB_SYS_TABLES, INNODB_BUFFER_PAGE, ecc.) insieme a tabelle applicative rilevanti come *guestbook* e *users*, quest'ultima evidenziata in blu nella ricerca: è proprio la tabella che l'attaccante prenderà di mira nel passo successivo.

Cosa farebbe per l'aggressore il comando modificato di 1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'?

Questo comando modificato interrogherebbe INFORMATION_SCHEMA.columns invece di INFORMATION_SCHEMA.tables, e grazie alla clausola WHERE table_name='users' filtrerebbe il risultato mostrando solo i nomi delle **colonne della tabella users** (ad esempio user_id, first_name, last_name, user, password, avatar...), invece dell'elenco completo — molto più lungo e poco mirato — di tutte le tabelle del database. È il passo logico successivo dell'attaccante prima di estrarre i dati veri e propri: prima trova la tabella interessante, poi ne scopre la struttura esatta delle colonne.

Parte 6: L'attacco di SQL Injection si conclude

Nell'ultima fase l'attaccante usa finalmente la query 1' or 1=1 union select user, password from users# per estrarre nomi utente e hash delle password direttamente dalla tabella users.

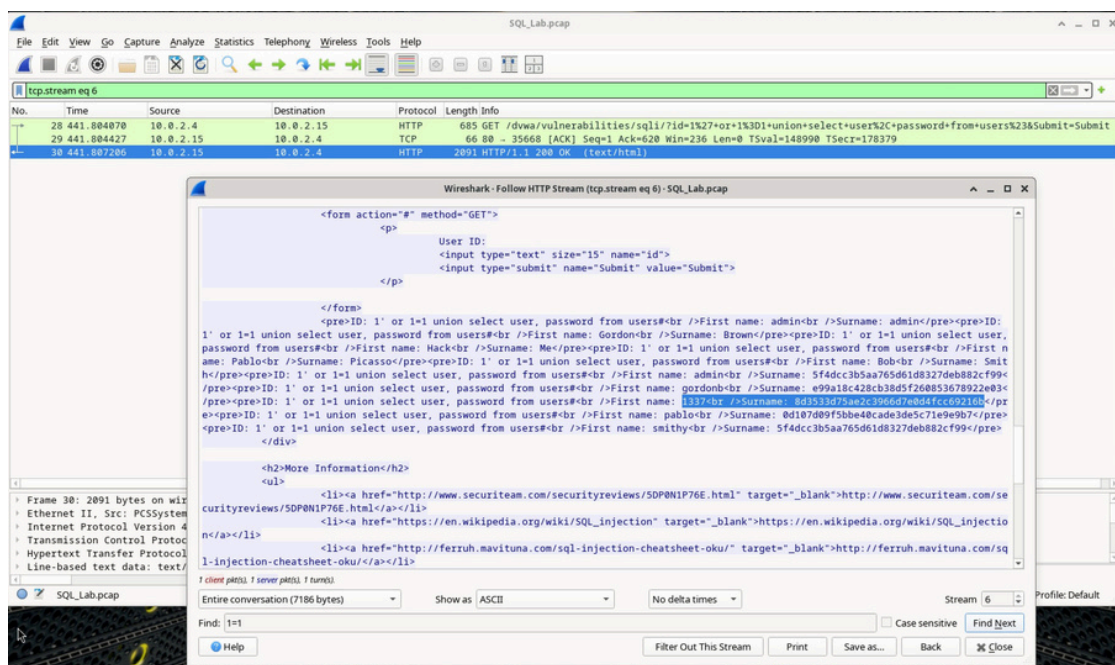


Figura 4.7 — Flusso HTTP finale (tcp.stream eq 6): l'output restituisce coppie utente/hash MD5, tra cui admin / 5f4dcc3b5aa765d61d8327deb882cf99, gordonb / e99a18c428cb38d5f260853678922e03, 1337 / 8d3533d75ae2c3966d7e0d4fcc69216b, pablo / 0d107d09f5bbe40cade3de5c71e9e9b7e e smithy / 5f4dcc3b5aa765d61d8327deb882cf99.

Quale utente ha l'hash della password di 8d3533d75ae2c3966d7e0d4fcc69216b?

L'hash 8d3533d75ae2c3966d7e0d4fcc69216b appartiene all'utente **1337**.

Qual è la password in chiaro? (decifrata con un hash cracker MD5 come crackstation.net)

Si tratta di un hash MD5 non salato, quindi facilmente reversibile tramite tabelle rainbow/dizionari online. La password in chiaro corrispondente è **charley**. Questo conferma quanto sia pericoloso l'uso di MD5 senza salt per memorizzare le password: bastano pochi secondi su un servizio gratuito per recuperare la password originale a partire dal solo hash.

Domanda di Riflessione 1 — Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

I siti web sono comunemente basati su database relazionali e utilizzano il linguaggio SQL per interrogarli. Se l'input fornito dall'utente non viene validato/sanificato prima di essere inserito in una query, un aggressore può alterare la logica della query stessa (come ho potuto osservare passo dopo passo in questo laboratorio), arrivando a leggere dati riservati, falsificare la propria identità (bypassando l'autenticazione), modificare o cancellare dati nel database e, in scenari ancora più gravi, ottenere l'esecuzione di comandi sul sistema operativo sottostante. La gravità concreta di un attacco SQL Injection dipende sia dalle competenze dell'aggressore sia dai privilegi con cui l'applicazione si connette al database: in questo caso, connettendosi come root, anche un singolo parametro vulnerabile ha esposto l'intero database.

Domanda di Riflessione 2 — Quali sono 2 metodi o passaggi che possono essere adottati per prevenire gli attacchi di SQL injection?

Dalla mia ricerca, i due metodi più efficaci ed effettivamente utilizzati in produzione sono:

- 1) l'uso sistematico di **query parametrizzate / prepared statement**, che separano nettamente il codice SQL dai dati forniti dall'utente, rendendo impossibile per l'input alterare la struttura logica della query (a differenza della concatenazione diretta di stringhe usata nell'applicazione DVWA analizzata);
- 2) l'applicazione del **principio del privilegio minimo** sull'account di connessione al database: l'utenza usata dall'applicazione dovrebbe avere solo i permessi strettamente necessari (SELECT/INSERT/UPDATE sulle sole tabelle richieste) e mai privilegi amministrativi come root, in modo che anche in caso di SQL Injection riuscita il danno resti contenuto. A questi si possono aggiungere ulteriori livelli di difesa come l'input validation/whitelisting, un Web Application Firewall (WAF) e il monitoraggio/logging delle query SQL anomale.